

Calculo Numérico Computacional - Practica Calificada 1

Resuelva los siguientes problemas mostrando todos los pasos.

1. (10 points) Determine intervalos que contengan soluciones reales de las siguientes ecuaciones:

- (a) $x - 3^{-x} = 0$
- (b) $4x^2 - e^x = 0$
- (c) $x^3 - 2x^2 - 4x + 3 = 0$

Solution: Para cada función $f(x)$, buscamos intervalos $[a, b]$ tales que:

$$f(a) \cdot f(b) < 0$$

lo que garantiza (por el **Teorema del Valor Intermedio**) la existencia de al menos una raíz en (a, b) .

a). $f(x) = x - 3^{-x}$, evaluamos:

$$f(0) = 0 - 1 = -1 < 0$$

$$f(1) = 1 - \frac{1}{3} = \frac{2}{3} > 0$$

Como hay cambio de signo: existe una raíz en $(0, 1)$

b). $f(x) = 4x^2 - e^x$, evaluamos en varios puntos:

$$f(0) = 0 - 1 = -1 < 0$$

$$f(1) = 4 - e \approx 4 - 2.718 = 1.282 > 0$$

Como hay cambio de signo: existe una raíz en $(0, 1)$. Ahora buscamos otra raíz:

$$f(2) = 16 - e^2 \approx 16 - 7.389 = 8.611 > 0$$

$$f(3) = 36 - e^3 \approx 36 - 20.085 = 15.915 > 0$$

$$f(4) = 64 - e^4 \approx 64 - 54.598 = 9.402 > 0$$

$$f(5) = 100 - e^5 \approx 100 - 148.41 = -48.41 < 0$$

Como hay cambio de signo: existe otra raíz en $(4, 5)$.

c). $f(x) = x^3 - 2x^2 - 4x + 3$, evaluamos:

$$f(-2) = -8 - 8 + 8 + 3 = -5 < 0$$

$$f(-1) = -1 - 2 + 4 + 3 = 4 > 0$$

Como hay cambio de signo: existe una raíz en $(-2, -1)$.

$$f(0) = 3 > 0$$

$$f(1) = 1 - 2 - 4 + 3 = -2 < 0$$

Como hay cambio de signo: existe una raíz en $(0, 1)$.

$$f(2) = 8 - 8 - 8 + 3 = -5 < 0$$

$$f(3) = 27 - 18 - 12 + 3 = 0$$

encontramos $f(x) = 0$, por lo tanto existe una raíz en $x = 3$

2. (10 points) Usando la definición de suma de Riemann, evalúe la integral definida:

$$\int_0^2 (3x^2 + 2x) dx$$

Considere una partición uniforme del intervalo $[0, 2]$ con $n = 5$ subintervalos y utilice puntos extremos derechos.

1. Expresar la suma de Riemann correspondiente.
2. Simplifique la suma en términos de n .
3. Evalúe el límite cuando $n \rightarrow \infty$.

Solution:

Partición del intervalo Se divide $[0, 2]$ en n subintervalos uniformes:

$$\Delta x = \frac{2 - 0}{n} = \frac{2}{n} \Rightarrow x_i = 0 + i\Delta x = \frac{2i}{n}, \quad i = 1, 2, \dots, n$$

Suma de Riemann

$$\begin{aligned} S_n &= \sum_{i=1}^n f(x_i) \Delta x \\ &= \sum_{i=1}^n \left[3 \left(\frac{2i}{n} \right)^2 + 2 \left(\frac{2i}{n} \right) \right] \frac{2}{n} \\ &= \sum_{i=1}^n \left[3 \frac{4i^2}{n^2} + \frac{4i}{n} \right] \frac{2}{n} = \sum_{i=1}^n \left(\frac{12i^2}{n^2} + \frac{4i}{n} \right) \frac{2}{n} \\ S_n &= \sum_{i=1}^n \left(\frac{24i^2}{n^3} + \frac{8i}{n^2} \right) \end{aligned}$$

Separando sumas:

$$S_n = \frac{24}{n^3} \sum_{i=1}^n i^2 + \frac{8}{n^2} \sum_{i=1}^n i$$

Fórmulas conocidas

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}, \quad \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

Sustituyendo:

$$\begin{aligned} S_n &= \frac{24}{n^3} \cdot \frac{n(n+1)(2n+1)}{6} + \frac{8}{n^2} \cdot \frac{n(n+1)}{2} \\ &= \frac{4}{n^3} n(n+1)(2n+1) + \frac{4}{n^2} n(n+1) \\ S_n &= \frac{4(n+1)(2n+1)}{n^2} + \frac{4(n+1)}{n} \end{aligned}$$

Analizando la expansión

$$(n+1)(2n+1) = 2n^2 + 3n + 1$$

$$\begin{aligned} S_n &= \frac{4(2n^2 + 3n + 1)}{n^2} + \frac{4(n+1)}{n} \\ &= 4 \left(2 + \frac{3}{n} + \frac{1}{n^2} \right) + 4 \left(1 + \frac{1}{n} \right) = 8 + \frac{12}{n} + \frac{4}{n^2} + 4 + \frac{4}{n} \\ S_n &= 12 + \frac{16}{n} + \frac{4}{n^2} \end{aligned}$$

Límite cuando $n \rightarrow \infty$

$$\lim_{n \rightarrow \infty} S_n = \lim_{n \rightarrow \infty} \left(12 + \frac{16}{n} + \frac{4}{n^2} \right) = 12$$

Resultado final

$$\boxed{\int_0^2 (3x^2 + 2x) dx = 12}$$

3. (10 points) Considere el siguiente sistema de ecuaciones lineales:

$$\begin{cases} 2x + y - z = 1 \\ -3x - y + 2z = -4 \\ -2x + y + 2z = -3 \end{cases}$$

1. Escriba el sistema en forma matricial $A\mathbf{x} = \mathbf{b}$.
2. Resuelva el sistema utilizando el método de eliminación de Gauss.
3. Calcule la matriz inversa A^{-1} .
4. Verifique la solución mediante $\mathbf{x} = A^{-1}\mathbf{b}$.

Solution:

Forma matricial El sistema se escribe como:

$$A\mathbf{x} = \mathbf{b}$$

$$A = \begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ -4 \\ -3 \end{bmatrix}$$

Método de eliminación de Gauss

Matriz aumentada:

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 1 \\ -3 & -1 & 2 & -4 \\ -2 & 1 & 2 & -3 \end{array} \right]$$

Paso 1: Hacer pivote en (1,1)

$$R_2 \leftarrow R_2 + \frac{3}{2}R_1, \quad R_3 \leftarrow R_3 + R_1$$

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 1 \\ 0 & 0.5 & 0.5 & -2.5 \\ 0 & 2 & 1 & -2 \end{array} \right]$$

Paso 2: Eliminar debajo del pivote (2,2)

$$R_3 \leftarrow R_3 - 4R_2$$

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 1 \\ 0 & 0.5 & 0.5 & -2.5 \\ 0 & 0 & -1 & 8 \end{array} \right]$$

Sustitución hacia atrás

$$\left. \begin{array}{l} -z = 8 \Rightarrow z = -8 \\ 0.5y + 0.5(-8) = -2.5 \Rightarrow 0.5y - 4 = -2.5 \Rightarrow y = 3 \\ 2x + y - z = 1 \Rightarrow 2x + 3 + 8 = 1 \Rightarrow 2x = -10 \Rightarrow x = -5 \end{array} \right\} \Rightarrow \mathbf{x} = \begin{bmatrix} -5 \\ 3 \\ -8 \end{bmatrix}$$

Cálculo de la inversa A^{-1} Se construye la matriz aumentada:

$$\left[\begin{array}{ccc|ccc} 2 & 1 & -1 & 1 & 0 & 0 \\ -3 & -1 & 2 & 0 & 1 & 0 \\ -2 & 1 & 2 & 0 & 0 & 1 \end{array} \right]$$

Tras aplicar eliminación de Gauss-Jordan (pasos omitidos por brevedad):

$$A^{-1} = \begin{bmatrix} 4 & 3 & -1 \\ -2 & -2 & 1 \\ 3 & 2 & -1 \end{bmatrix}$$

Verificación: $\mathbf{x} = A^{-1}\mathbf{b}$

$$\mathbf{x} = \begin{bmatrix} 4 & 3 & -1 \\ -2 & -2 & 1 \\ 3 & 2 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ -4 \\ -3 \end{bmatrix} = \begin{bmatrix} 4(1) + 3(-4) + (-1)(-3) \\ -2(1) + (-2)(-4) + (1)(-3) \\ 3(1) + 2(-4) + (-1)(-3) \end{bmatrix} = \begin{bmatrix} -5 \\ 3 \\ -8 \end{bmatrix}$$

Conclusión

- El sistema tiene solución única.
- La eliminación de Gauss y la inversa producen el mismo resultado.
- La matriz es invertible ($\det(A) \neq 0$).

4. (10 points) Usando la definición de derivada, calcule:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

para la función:

$$f(x) = x^3 - 2x^2 + x$$

1. Sustituya en la definición de derivada.
2. Expanda completamente $f(x+h)$.
3. Simplifique el cociente incremental.
4. Evalúe el límite.

Solution: Sustitución en la definición de derivada

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} = \lim_{h \rightarrow 0} \frac{(x+h)^3 - 2(x+h)^2 + (x+h) - (x^3 - 2x^2 + x)}{h}$$

Expansión de $f(x+h)$

$$(x+h)^3 = x^3 + 3x^2h + 3xh^2 + h^3$$

$$(x+h)^2 = x^2 + 2xh + h^2$$

Sustituyendo:

$$\begin{aligned} f(x+h) &= x^3 + 3x^2h + 3xh^2 + h^3 - 2(x^2 + 2xh + h^2) + x + h \\ &= x^3 + 3x^2h + 3xh^2 + h^3 - 2x^2 - 4xh - 2h^2 + x + h \end{aligned}$$

Sustitución en el cociente incremental

$$\begin{aligned} f(x+h) - f(x) &= (x^3 + 3x^2h + 3xh^2 + h^3 - 2x^2 - 4xh - 2h^2 + x + h) - (x^3 - 2x^2 + x) \\ &= x^3 + 3x^2h + 3xh^2 + h^3 - 2x^2 - 4xh - 2h^2 + x + h - x^3 + 2x^2 - x \\ &= 3x^2h + 3xh^2 + h^3 - 4xh - 2h^2 + h \\ &= h(3x^2 + 3xh + h^2 - 4x - 2h + 1) \\ \frac{f(x+h) - f(x)}{h} &= 3x^2 + 3xh + h^2 - 4x - 2h + 1 \end{aligned}$$

Evaluación del límite

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} (3x^2 + 3xh + h^2 - 4x - 2h + 1) \\ f'(x) &= 3x^2 - 4x + 1 \end{aligned}$$

5. (10 points) Considere la función:

$$f(x) = x^4 - 4x^3 + 6x^2 - 2x$$

1. Calcule la primera derivada $f'(x)$.
2. Determine los puntos críticos resolviendo $f'(x) = 0$.
3. Calcule la segunda derivada $f''(x)$.
4. Clasifique cada punto crítico como máximo local, mínimo local o punto de inflexión usando el criterio de la segunda derivada.
5. Determine los intervalos donde la función es creciente y decreciente.
6. (Opcional) Grafique la función y verifique sus resultados.

Solution: Primera derivada

$$f'(x) = \frac{d}{dx}(x^4 - 4x^3 + 6x^2 - 2x) = 4x^3 - 12x^2 + 12x - 2$$

Puntos críticos: $f'(x) = 0$

$$4x^3 - 12x^2 + 12x - 2 = 0$$

$$2(2x^3 - 6x^2 + 6x - 1) = 0$$

$$2x^3 - 6x^2 + 6x - 1 = 0$$

Probamos $x = 1$:

$$2(1)^3 - 6(1)^2 + 6(1) - 1 = 2 - 6 + 6 - 1 = 1 \neq 0$$

Probamos $x = \frac{1}{2}$:

$$2\left(\frac{1}{8}\right) - 6\left(\frac{1}{4}\right) + 6\left(\frac{1}{2}\right) - 1 = \frac{1}{4} - \frac{3}{2} + 3 - 1 = \frac{3}{4} \neq 0$$

Se resuelve numéricamente (o factorización conocida):

$$f'(x) = 2(2x - 1)(x - 1)^2 \Rightarrow \begin{cases} x = \frac{1}{2} \\ x = 1 \end{cases}$$

Segunda derivada

$$f''(x) = \frac{d}{dx}(4x^3 - 12x^2 + 12x - 2) = 12x^2 - 24x + 12 = 12(x - 1)^2$$

Clasificación de puntos críticos

- Para $x = \frac{1}{2}$:

$$f''\left(\frac{1}{2}\right) = 12\left(\frac{1}{2} - 1\right)^2 = 12\left(-\frac{1}{2}\right)^2 = 12 \cdot \frac{1}{4} = 3 > 0 \Rightarrow \text{Mínimo local en } x = \frac{1}{2}$$

- Para $x = 1$:

$$f''(1) = 12(1 - 1)^2 = 0$$

El criterio no es concluyente. Revisamos cambio de signo de $f'(x)$:

$$f'(x) = 2(2x - 1)(x - 1)^2$$

$(x - 1)^2 \geq 0$, no cambia de signo, entonces el signo depende de $2x - 1$. Alrededor de $x = 1$, $2x - 1 > 0$, por lo tanto:

$$f'(x) > 0 \quad \text{a ambos lados} \Rightarrow \text{Punto de inflexión en } x = 1$$

Intervalos de crecimiento y decrecimiento

$$f'(x) = 2(2x - 1)(x - 1)^2$$

- Si $x < \frac{1}{2}$: $2x - 1 < 0 \Rightarrow f'(x) < 0 \rightarrow$ decreciente
- Si $x > \frac{1}{2}$: $2x - 1 > 0 \Rightarrow f'(x) > 0 \rightarrow$ creciente

Entonces:

Decrece en $(-\infty, \frac{1}{2})$

Crece en $(\frac{1}{2}, \infty)$

Valores de la función

$$f\left(\frac{1}{2}\right) = \frac{1}{16} - \frac{4}{8} + \frac{6}{4} - 1 = \frac{1}{16} - \frac{1}{2} + \frac{3}{2} - 1 = \frac{1}{16}$$

$$f(1) = 1 - 4 + 6 - 2 = 1$$

Resultado final

- Mínimo local en:

$$\left(\frac{1}{2}, \frac{1}{16}\right)$$

- Punto de inflexión en:

$$(1, 1)$$

- Decrece en:

$$(-\infty, \frac{1}{2})$$

- Crece en:

$$(\frac{1}{2}, \infty)$$

Parte II: Problemas de Programacion en Python o C/C++

1. (25 points) Escriba un programa que lea un archivo de texto llamado `datos.txt`, el cual contiene una lista de números reales (uno por línea).

1. Calcule la media o promedio(μ) , mediana y desviación estándar σ . mediana:

- Ejemplo A (impar): Datos: 4, 6, 7, 7, 11. La mediana es 7.
- Ejemplo B (par): Datos: 3, 4, 4, 6, 7, 9.
 - Ordenados: 3, 4, 4, 6, 7, 9.
 - Valores centrales: 4 y 6.
 - Mediana: $(4 + 6)/2$

$$\text{desviación estándar } \sigma = \sqrt{\frac{\sum (x_i - \mu)^2}{N}}$$

2. Identifique los valores atípicos (outliers) usando el criterio de 2σ (o dos desviaciones estándar). Fórmula del Criterio

- Límite Superior: $media + 2\sigma$
- Límite Inferior: $media - 2\sigma$

Un valor es atípico si:

$$\sigma > \text{Límite Superior} \quad \text{o} \quad \sigma < \text{Límite Inferior}$$

3. Genere un archivo `reporte.txt` con los resultados.

4. (Opcional) Grafique un histograma de los datos.

Restricciones:

- No usar librerías estadísticas avanzadas (solo `math` o `numpy` básico).

- Manejar posibles errores de lectura del archivo.

Solution:

Objetivo: Leer datos desde un archivo, calcular estadísticas básicas, detectar outliers y generar un reporte.

1. Pseudocódigo

```

1 INICIO
2   Abrir archivo "datos.txt"
3   SI no se puede abrir:
4     Mostrar error y terminar
5
6   Leer todos los numeros en una lista
7
8   N = tamaño de la lista
9
10  Calcular media:
11    suma = 0
12    PARA cada x en lista:
13      suma += x
14    media = suma / N
15
16  Ordenar la lista
17
18  Calcular mediana:
19    SI N es impar:
20      mediana = lista[N/2]
21    SI N es par:
22      mediana = (lista[N/2 - 1] + lista[N/2]) / 2
23
24  Calcular desviacion estandar:
25    suma2 = 0
26    PARA cada x en lista:
27      suma2 += (x - media)^2
28    sigma = sqrt(suma2 / N)
29
30  Calcular limites:
31    LI = media - 2*sigma
32    LS = media + 2*sigma
33
34  Detectar outliers:
35    PARA cada x en lista:
36      SI x < LI o x > LS:
37        marcar como outlier
38
39  Crear archivo "reporte.txt"
40  Escribir media, mediana, sigma
41  Escribir limites
42  Escribir lista de outliers
43
44 FIN

```

2. Programa en C++

```

1 #include <iostream>
2 #include <fstream>
3 #include <vector>
4 #include <algorithm>
5 #include <cmath>
6
7 using namespace std;
8
9 int main() {
10   ifstream file("datos.txt");
11   if (!file) {
12     cout << "Error al abrir el archivo." << endl;
13     return 1;
14   }
15
16   vector<double> datos;
17   double x;
18
19   while (file >> x) {
20     datos.push_back(x);
21   }
22
23   int N = datos.size();
24   if (N == 0) {
25     cout << "Archivo vacio." << endl;
26     return 1;
27   }
28
29   // Media
30   double suma = 0;
31   for (double v : datos) suma += v;
32   double media = suma / N;
33
34   // Mediana
35   sort(datos.begin(), datos.end());
36   double mediana;
37   if (N % 2 == 0)
38     mediana = (datos[N/2 - 1] + datos[N/2]) / 2.0;
39   else
40     mediana = datos[N/2];
41
42   // Desviacion estandar
43   double suma2 = 0;
44   for (double v : datos)
45     suma2 += pow(v - media, 2);
46   double sigma = sqrt(suma2 / N);
47
48   // Limites
49   double LI = media - 2*sigma;
50   double LS = media + 2*sigma;
51
52   // Outliers
53   vector<double> outliers;
54   for (double v : datos) {
55     if (v < LI || v > LS)
56       outliers.push_back(v);
57   }
58
59   // Archivo de salida
60   ofstream out("reporte.txt");
61
62   out << "Media: " << media << endl;
63   out << "Mediana: " << mediana << endl;
64   out << "Desviacion estandar: " << sigma << endl;
65
66   out << "Limite inferior: " << LI << endl;
67   out << "Limite superior: " << LS << endl;
68
69   out << "Outliers:" << endl;
70   for (double v : outliers)
71     out << v << endl;
72
73   out.close();
74
75   cout << "Reporte generado correctamente." << endl;
76   return 0;
77 }

```

3.

Observaciones importantes

- Se usa desviación estándar poblacional (divide entre N).
- El criterio correcto de outliers es:

$$x < \mu - 2\sigma \quad \text{o} \quad x > \mu + 2\sigma$$

(no comparar σ como indica el enunciado).

- Se incluye manejo básico de errores (archivo vacío o inexistente).

- (25 points) Desarrolle un programa que gestione una matriz cuadrada de tamaño $N \times N$ (con $N \leq 100$), ingresada por el usuario.

Restricciones:

1. Lea la matriz desde la entrada estándar.
 2. Verifique si la matriz es **simétrica** (es decir, $A_{ij} = A_{ji}$).
 3. Calcule la suma de los elementos de:
 - la diagonal principal,
 - la diagonal secundaria.
 4. Encuentre el valor máximo y mínimo de toda la matriz junto con sus posiciones.
 5. Genere la matriz transpuesta y muéstrela.
- No usar librerías externas.
 - Usar estructuras básicas del lenguaje (arreglos, ciclos, condicionales).
 - Optimizar el uso de memoria.

Salida esperada:

- Indicar si la matriz es simétrica o no.
- Mostrar las sumas de diagonales.
- Mostrar el valor máximo y mínimo con sus posiciones.
- Mostrar la matriz transpuesta.

Solution:

Pseudocódigo

```
1 Inicio
2   Leer N
3   Si N > 100 entonces terminar
4
5   Definir matriz A[N][N]
6
7   // Lectura
8   Para i = 0 hasta N-1
9     Para j = 0 hasta N-1
10      Leer A[i][j]
11
12   // Verificar simetria
13   esSimetrica = verdadero
14   Para i = 0 hasta N-1
15     Para j = 0 hasta N-1
16       Si A[i][j] != A[j][i]
17         esSimetrica = falso
18
19   // Diagonales
20   sumaPrincipal = 0
21   sumaSecundaria = 0
22   Para i = 0 hasta N-1
23     sumaPrincipal += A[i][i]
24     sumaSecundaria += A[i][N-1-i]
```

```
1 // Maximo y minimo
2 max = A[0][0]
3 min = A[0][0]
4 posMax = (0,0)
5 posMin = (0,0)
6
7 Para i = 0 hasta N-1
8   Para j = 0 hasta N-1
9     Si A[i][j] > max
10      max = A[i][j]
11      posMax = (i,j)
12     Si A[i][j] < min
13      min = A[i][j]
14      posMin = (i,j)
15
16 // Transpuesta
17 Para i = 0 hasta N-1
18   Para j = 0 hasta N-1
19     T[i][j] = A[j][i]
20
21 // Salida
22 Mostrar si es simetrica
23 Mostrar sumas de diagonales
24 Mostrar max, min y posiciones
25 Mostrar matriz transpuesta
26 Fin
```

Programa en C++

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5   int N;
6   cout << "Ingrese N: ";
7   cin >> N;
8
9   if (N > 100) {
10    cout << "N excede el limite." << endl;
11    return 0;
12  }
13
14  int A[100][100];
15
16  // Lectura de matriz
17  cout << "Ingrese la matriz:" << endl;
18  for (int i = 0; i < N; i++)
19    for (int j = 0; j < N; j++)
20      cin >> A[i][j];
21
22  // Verificar simetria
23  bool esSimetrica = true;
24  for (int i = 0; i < N; i++)
25    for (int j = 0; j < N; j++)
26      if (A[i][j] != A[j][i])
27        esSimetrica = false;
28
29  // Sumas diagonales
30  int sumaPrincipal = 0, sumaSecundaria = 0;
31  for (int i = 0; i < N; i++) {
32    sumaPrincipal += A[i][i];
33    sumaSecundaria += A[i][N-1-i];
34  }
35
36  // Maximo y minimo
37  int maxVal = A[0][0], minVal = A[0][0];
38  int iMax = 0, jMax = 0, iMin = 0, jMin = 0;
```

```
1 for (int i = 0; i < N; i++) {
2   for (int j = 0; j < N; j++) {
3     if (A[i][j] > maxVal) {
4       maxVal = A[i][j];
5       iMax = i; jMax = j;
6     }
7     if (A[i][j] < minVal) {
8       minVal = A[i][j];
9       iMin = i; jMin = j;
10    }
11  }
12 }
13
14 // Salida
15 if (esSimetrica)
16   cout << "La matriz es simetrica" << endl;
17 else
18   cout << "La matriz NO es simetrica" << endl;
19
20 cout << "Suma diagonal principal: " << sumaPrincipal << endl;
21 cout << "Suma diagonal secundaria: " << sumaSecundaria << endl;
22
23 cout << "Maximo: " << maxVal
24     << " en (" << iMax << ", " << jMax << ")" << endl;
25
26 cout << "Minimo: " << minVal
27     << " en (" << iMin << ", " << jMin << ")" << endl;
28
29 // Transpuesta (sin matriz extra)
30 cout << "Matriz transpuesta:" << endl;
31 for (int i = 0; i < N; i++) {
32   for (int j = 0; j < N; j++) {
33     cout << A[j][i] << " ";
34   }
35   cout << endl;
36 }
37
38 return 0;
39 }
```


Observaciones

- No se usan librerías externas.
- Se optimiza memoria mostrando la transpuesta sin almacenarla.
- Complejidad: $O(N^2)$.

Calculo Numérico Computacional - Practica Calificada 1

Apellidos y Nombres: _____ ∞

Responda las siguientes preguntas teóricas de opción múltiple utilizando su mejor criterio y conocimiento para seleccionar la respuesta correcta.

Parte III: Preguntas de Opción Múltiple, Python

1. (1 point) ¿Qué imprime el siguiente código?

```
1 a = [1, 2, 3]
2 b = a
3 b[0] = 10
4 print(a)
```

- A. [1,2,3]
- B. [10,2,3]**
- C. Error
- D. Ninguna

2. (1 point) ¿Cuál es el resultado de `bool([])`?

- A. True
- B. False**
- C. Error
- D. None

3. (1 point) ¿Qué imprime?

```
1 print(type((1)))
```

- A. int**
- B. tuple
- C. list
- D. error

4. (1 point) ¿Qué hace `set([1,1,2,2,3])`?

- A. Error
- B. Elimina duplicados**
- C. Ordena
- D. Nada

5. (1 point) ¿Resultado de `"abc".upper()`?

- A. abc
- B. ABC**
- C. error
- D. Abc

6. (1 point) ¿Qué imprime?

```
1 x = [1,2,3]
2 print(x[-1])
```

- A. 1
- B. 2
- C. 3**
- D. Error

7. (1 point) ¿Resultado de `sum([1,2,3])`?

- A. 5
- B. 6**
- C. Error
- D. None

8. (1 point) ¿Qué imprime?

```
1 for i in range(3):
2     pass
3 print(i)
```

- A. 0
- B. 1
- C. 2**
- D. Error

9. (1 point) ¿Qué hace `dict.get("x",0)`?

- A. Error
- B. Retorna valor o 0 si no existe**
- C. Elimina clave
- D. Inserta valor

10. (1 point) ¿Qué imprime?

```
1 print(3 == 3.0)
```

- A. True**
- B. False
- C. Error
- D. None

Parte IV: Preguntas de Opción Múltiple, C/C++

1. (1 point) ¿Qué imprime?

```
1 int x=3;
2 cout<<x++;
```

- A. 3**
- B. 4
- C. Error
- D. Ninguno

2. (1 point) ¿Qué imprime?

```
1 int a=5;
2 int b=2;
3 cout<<a%b;
```

- A. 3
- B. 1**
- C. 2
- D. Error

3. (1 point) ¿Qué tipo devuelve sizeof(int)?

- A. int
- B. size_t**
- C. float
- D. char

4. (1 point) ¿Qué hace break?

- A. Continúa
- B. Sale del bucle**
- C. Repite
- D. Error

5. (1 point) ¿Qué imprime?

```
1 int x=0;
2 while(x<3){
3     cout<<x;
4     x++;
5 }
```

- A. 012**
- B. 123

C. 321

D. Error

6. (1 point) ¿Qué significa &x?

- A. valor
- B. dirección de memoria**
- C. puntero
- D. referencia

7. (1 point) ¿Qué imprime?

```
1 int x=5;
2 cout<<(x>3 && x<10);
```

- A. 1**
- B. 0
- C. true
- D. error

8. (1 point) ¿Qué hace return 0;?

- A. Error
- B. Finaliza programa correctamente**
- C. Repite
- D. Nada

9. (1 point) ¿Qué imprime?

```
1 int x=2;
2 cout<<x*3+1;
```

- A. 7
- B. 7**
- C. 6
- D. Error

10. (1 point) ¿Qué es un puntero?

- A. variable normal
- B. variable que almacena dirección**
- C. función
- D. constante